



JAVA BASICS



~ Handwritten Style Notes ~

Variables

Loops

Arrays

OOP Intro

Operators

Methods

Strings

JVM · JRE · JDK

What's Inside

- Introduction to Java & its Features
- JDK, JRE, JVM explained with diagram
- How a Java program runs (Compilation flow)
- Variables, Data types & Type Casting
- Operators, Input handling (Scanner)
- Control statements – if/else, switch
- Loops – for, while, do-while
- Arrays, Strings, Methods
- Intro to Class & Object (OOP)
- Quick Recap Mind-map at the end

 Prepared by – **Akshit Sharma**

 akshbuilds.tech

📞 8510968871 / ✉ akshubhardwaj231@gmail.com

1 What is Java?

Java is a high-level, object-oriented programming language developed by James Gosling at Sun Microsystems (released 1995, now owned by Oracle). It is used to build everything from mobile apps (Android) to large enterprise systems.

💡 Golden Rule

"Write Once, Run Anywhere" (WORA)

2 Key Features of Java

Feature	Meaning
🔄 Platform Independent	Compiled code (bytecode) runs on any device with a JVM
📦 Object-Oriented	Everything is organized around objects & classes
🛡️ Robust	Strong memory management + exception handling
🔒 Secure	No explicit pointers, runs inside a sandboxed JVM
⚙️ Simple	Easy syntax, removed complex features like pointers
🧵 Multithreaded	Can perform many tasks at the same time
🗑️ Automatic Garbage Collection	Unused memory is cleaned automatically

📝 Note

Java is both **Compiled** and **Interpreted** — source code is compiled into **bytecode**, which the JVM then interprets/executes.

3

JDK vs JRE vs JVM

JVM

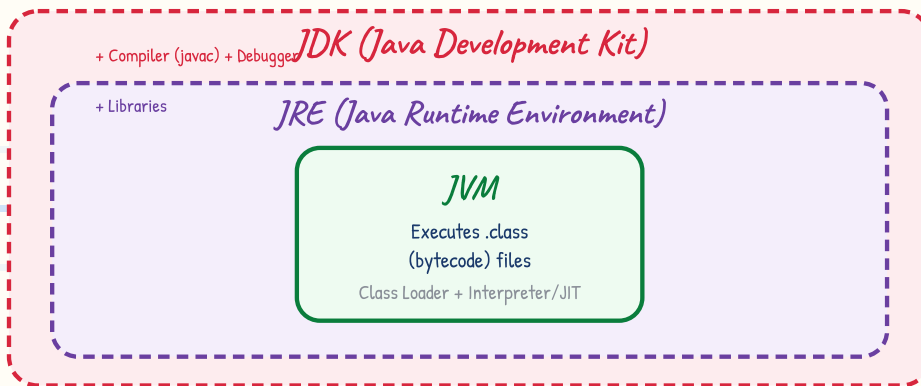
Java Virtual Machine – **executes** bytecode.
Converts it to machine code for your OS.

JRE

Java Runtime Environment = JVM +
Libraries. Needed to **run** Java programs.

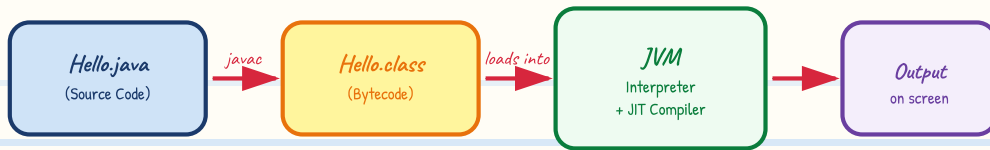
JDK

Java Development Kit = JRE + Compiler + Dev tools. Needed to **write & compile** Java programs.

**Remember**

Think of it like nesting dolls: **JDK** $\supset$ **JRE** $\supset$ **JVM**. To code, you need JDK. To just run a Java app, JRE is enough.

4 How a Java Program Runs



Steps

- Write code in a `.java` file.
- Compile using `javac` → produces platform-independent `bytecode` (.class file).
- Run using `java` command → JVM interprets/executes the bytecode.
- JVM is different for each OS, but bytecode is the same everywhere — that's how "Write Once Run Anywhere" works!

5 Structure of a Java Program

```
public class Hello {  
    // main method - entry point of program  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Part	Meaning
class Hello	Defines a class named Hello (file must be Hello.java)
public static void main	The starting point – JVM always looks for this method
String[] args	Command-line arguments passed to the program
System.out.println()	Prints text to the console with a new line

Note

static = belongs to class, not object (no need to create object to call it) | **void** = returns nothing | **public** = accessible from anywhere

6 Comments in Java

```
// Single line comment  
/* Multi  
   line comment */  
/** Documentation comment (used for JavaDocs) */
```

7

Variables & Data Types

Definition

A **variable** is a named container that stores a value. In Java, every variable must have a **data type** (Java is statically typed).

```
int age = 21; // datatype name = value;
```

Type	Size	Example
byte	1 byte	byte b = 10;
short	2 bytes	short s = 500;
int	4 bytes	int x = 10000;
long	8 bytes	long l = 100000L;
float	4 bytes	float f = 3.14f;
double	8 bytes	double d = 3.1415;
char	2 bytes	char c = 'A';
boolean	1 bit	boolean flag = true;

Primitive

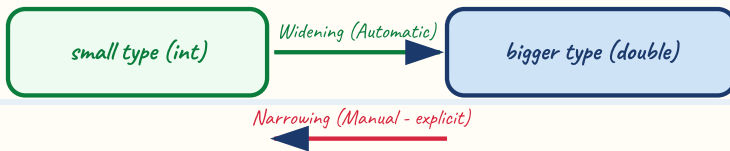
Stores actual value directly. e.g. int, char, boolean

Non-Primitive

Stores reference/address. e.g. String, Array, Object

8 Type Casting

Type Casting means converting one data type into another.



```
// Widening - done automatically, no data loss
int a = 10;
double b = a;    // int -> double

// Narrowing - must be done manually, may lose data
double x = 9.78;
int y = (int) x;  // y = 9 (decimal part lost)
```

⚠ Careful

Narrowing can cause **data loss** — always double-check when converting from a bigger type to a smaller one.

9 Operators in Java

Type	Operators	Example
Arithmetic	+ - * / %	a + b, a % b (remainder)
Relational	== != > < >= <=	a > b returns true/false
Logical	&& (AND) (OR) !(NOT)	(a>5 && b<10)
Assignment	= += -= *= /=	a += 5 → a = a + 5
Unary	++ -- + - !	a++ (post-increment)
Ternary	condition ? val1 : val2	max = (a>b) ? a : b

 ++ vs --

Pre-increment (++a): increases value first, then uses it. **Post-increment (a++):** uses value first, then increases it.

```
int a = 5;
System.out.println(a++); // prints 5, then a becomes 6
System.out.println(++a); // a becomes 7, then prints 7
```

10 Taking Input - Scanner class

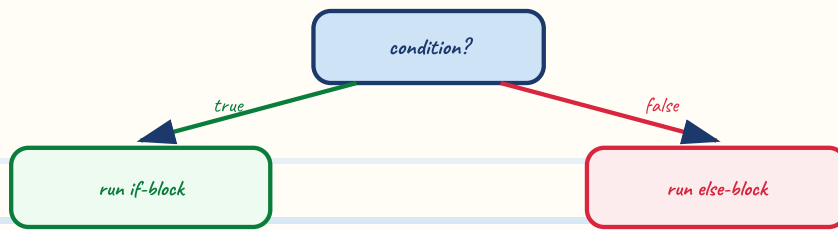
To take input from the user (keyboard), Java provides the **Scanner** class in `java.util` package.

```
import java.util.Scanner;

public class Demo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your age: ");
        int age = sc.nextInt(); // reads an integer
        System.out.println("You are " + age + " years old");
    }
}
```

Method	Reads
nextInt()	an integer
nextDouble()	a decimal number
next()	a single word (String)
nextLine()	a full line of text

11 Control Statements – if / else / switch

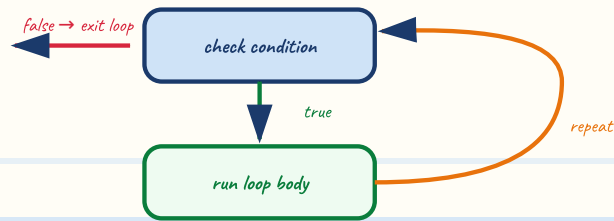


```
int marks = 75;
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 60) {
    System.out.println("Grade B"); // this runs
} else {
    System.out.println("Grade C");
}
```

switch statement

```
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Invalid");
}
```

12 Loops – for, while, do-while



```
// for loop
for(int i=1; i<=5; i++){
    System.out.println(i);
}
```

```
// while loop
int i=1;
while(i<=5){
    System.out.println(i);
    i++;
}
```

```
// do-while loop -> runs at LEAST once, condition checked at end
int i=1;
do {
    System.out.println(i);
    i++;
} while(i<=5);
```

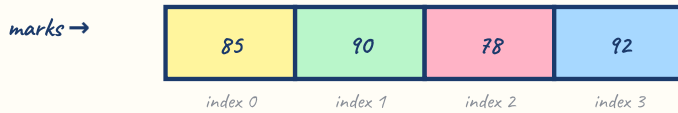
Which to use?

for → when count is known | **while** → when condition-based, count unknown | **do-while** → must execute body at least once (e.g. menus)

13 Arrays

An **array** stores multiple values of the **same data type** in one variable, in contiguous memory. Index starts from **0**.

```
int[] marks = {85, 90, 78, 92};  
System.out.println(marks[0]);    // 85 (first element)  
System.out.println(marks.length); // 4 (size of array)
```



```
// Looping through an array  
for(int i=0; i<marks.length; i++){  
    System.out.println(marks[i]);  
}
```

⚠ Careful

Accessing an index outside the array size (e.g. marks[4] above) throws **ArrayIndexOutOfBoundsException**.

14 Strings

A **String** is a sequence of characters. In Java, String is a **class** (non-primitive/object type), not a primitive.

```
String name = "Akshit";
System.out.println(name.length());           // 6
System.out.println(name.toUpperCase());       // AKSHIT
System.out.println(name.charAt(0));           // A
System.out.println(name.substring(0,3));      // Aks
System.out.println(name.equals("Akshit"));    // true
```

Method	What it does
length()	returns number of characters
toUpperCase() / toLowerCase()	changes case
charAt(i)	returns character at index i
substring(a,b)	extracts part of the string
equals()	compares two strings (use this, NOT ==)
concat() / +	joins two strings

⚠️ == vs equals()

Use **.equals()** to compare String content. **==** compares memory reference and can give wrong results!

15 Methods (Functions)

A **method** is a block of reusable code that performs a specific task, called by its name.

```
returnType methodName(parameters) {  
    // body  
    return value;  
}  
  
// Example  
static int add(int a, int b) {  
    return a + b;  
}  
  
public static void main(String[] args) {  
    int result = add(5, 3); // calling the method  
    System.out.println(result); // 8  
}
```



Why use methods?

Avoid repeating code, make programs organized/readable, and allow reusing logic anywhere in the program.

16 Class & Object – OOP Intro

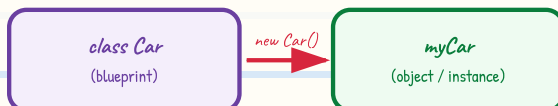
Class

A **blueprint/template** that defines properties (variables) & behaviours (methods). No memory is used until an object is created.

Object

A **real instance** of a class, created using the **new** keyword. Occupies actual memory.

```
class Car {  
    String color;           // property  
    void drive() {          // behaviour  
        System.out.println(color + " car is driving");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car();    // object created  
        myCar.color = "Red";  
        myCar.drive();            // Red car is driving  
    }  
}
```



Quick Recap

Java = OOP + Platform Independent

JDK → JRE → JVM

.java → javac → .class → JVM runs it

Variables need a data type

Widening = auto, Narrowing = manual cast

if/else & switch = decision making

for/while/do-while = repetition

Array = same-type values, index from 0

String → use .equals() not ==

Method = reusable block of code

Class = blueprint, Object = instance

Final Tip

Practice writing these small programs by hand on paper before typing – it builds real understanding of syntax and logic much faster than just reading notes!

Happy Coding! ☕

Prepared by – Akshit Sharma

🌐 akshbuilds.tech

📞 8510968871 / ✉ akshubhardwaj231@gmail.com